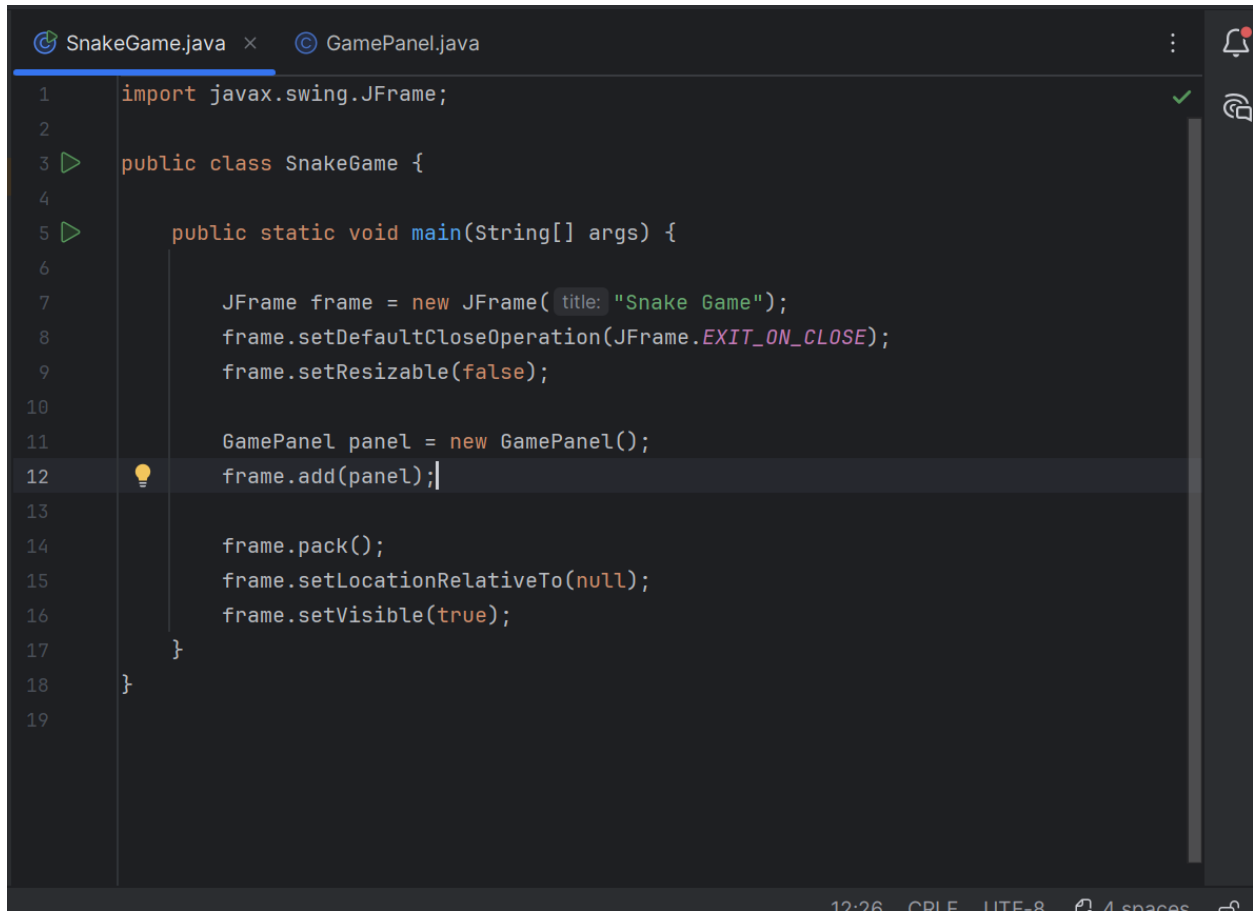


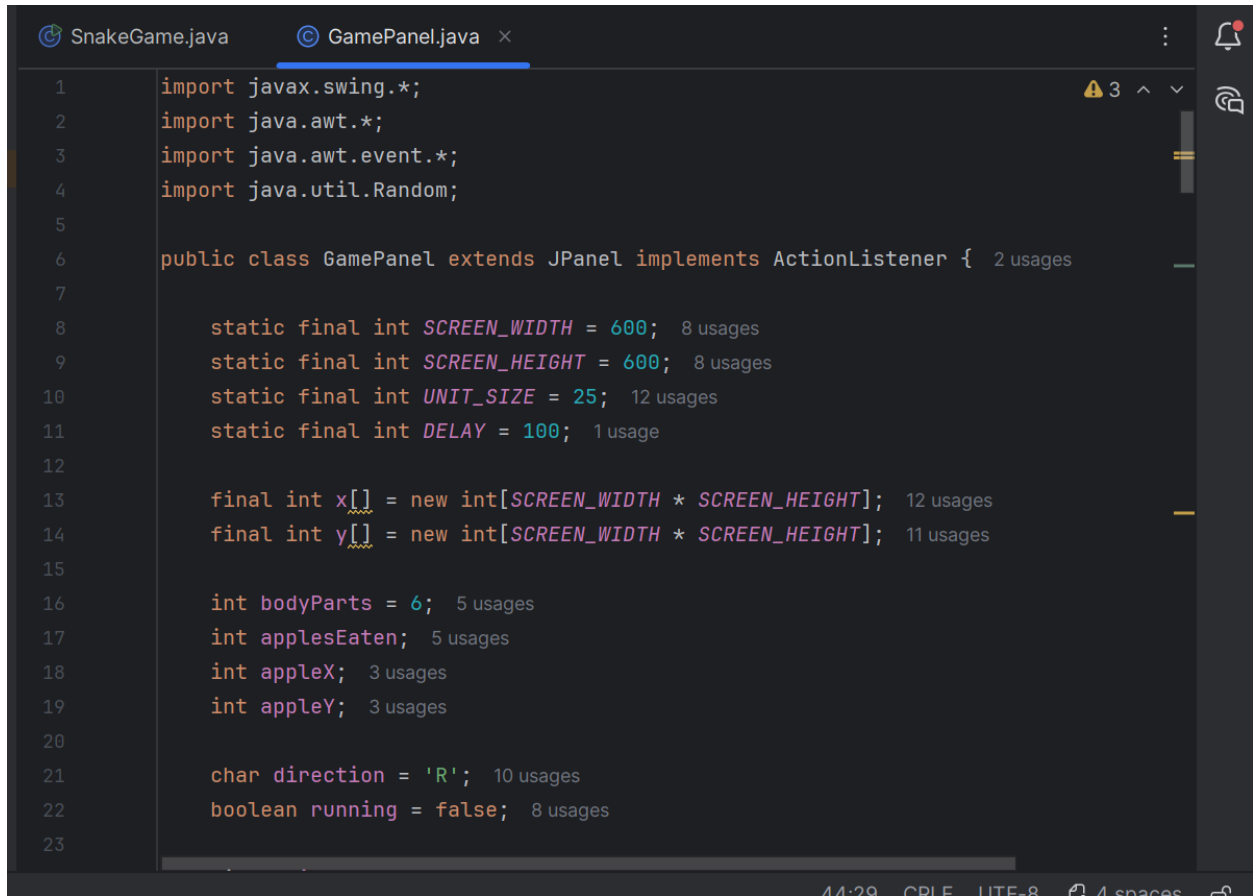
CODE SNIPPET:

MAIN CLASS



```
1  import javax.swing.JFrame;
2
3  public class SnakeGame {
4
5      public static void main(String[] args) {
6
7          JFrame frame = new JFrame( title: "Snake Game");
8          frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
9          frame.setResizable(false);
10
11          GamePanel panel = new GamePanel();
12          frame.add(panel);
13
14          frame.pack();
15          frame.setLocationRelativeTo(null);
16          frame.setVisible(true);
17      }
18  }
19
```

GAME PANEL



```
1  import javax.swing.*;
2  import java.awt.*;
3  import java.awt.event.*;
4  import java.util.Random;
5
6  public class GamePanel extends JPanel implements ActionListener { 2 usages
7
8      static final int SCREEN_WIDTH = 600; 8 usages
9      static final int SCREEN_HEIGHT = 600; 8 usages
10     static final int UNIT_SIZE = 25; 12 usages
11     static final int DELAY = 100; 1 usage
12
13     final int x[] = new int[SCREEN_WIDTH * SCREEN_HEIGHT]; 12 usages
14     final int y[] = new int[SCREEN_WIDTH * SCREEN_HEIGHT]; 11 usages
15
16     int bodyParts = 6; 5 usages
17     int applesEaten; 5 usages
18     int appleX; 3 usages
19     int appleY; 3 usages
20
21     char direction = 'R'; 10 usages
22     boolean running = false; 8 usages
23
```

```

21 char direction = 'R'; 10 usages
22 boolean running = false; 8 usages
23
24 Timer timer; 3 usages
25 Random random; 3 usages
26
27 GamePanel() { 1 usage
28     random = new Random();
29     this.setPreferredSize(new Dimension(SCREEN_WIDTH, SCREEN_HEIGHT));
30     this.setBackground(Color.black);
31     this.setFocusable(true);
32     this.addKeyListener(new MyKeyAdapter());
33     startGame();
34 }
35
36 public void startGame() { 2 usages
37     newApple();
38     running = true;
39     timer = new Timer(DELAY, listener: this);
40     timer.start();
41 }
42

```

```

42
43 @
44 public void paintComponent(Graphics g) {
45     super.paintComponent(g);
46     draw(g);
47 }
48
49 public void draw(Graphics g) { 1 usage
50
51     if (running) {
52
53         g.setColor(Color.red);
54         g.fillOval(appleX, appleY, UNIT_SIZE, UNIT_SIZE);
55
56         for (int i = 0; i < bodyParts; i++) {
57             if (i == 0)
58                 g.setColor(Color.green);
59             else
60                 g.setColor(new Color(r: 45, g: 180, b: 0));
61
62             g.fillRect(x[i], y[i], UNIT_SIZE, UNIT_SIZE);
63         }
64     }
65 }

```

```
64         g.setColor(Color.white);
65         g.setFont(new Font( name: "Arial", Font.BOLD, size: 30));
66         g.drawString( str: "Score: " + applesEaten, x: 20, y: 30);
67
68     } else {
69         gameOver(g);
70     }
71 }
72
73 public void newApple() { 2 usages
74     appleX = random.nextInt( bound: SCREEN_WIDTH / UNIT_SIZE) * UNIT_SIZE;
75     appleY = random.nextInt( bound: SCREEN_HEIGHT / UNIT_SIZE) * UNIT_SIZE;
76 }
77
78 public void move() { 1 usage
79
80     for (int i = bodyParts; i > 0; i--) {
81         x[i] = x[i - 1];
82         y[i] = y[i - 1];
83     }
84 }
```

```

85         switch (direction) {
86             case 'U':
87                 y[0] -= UNIT_SIZE;
88                 break;
89             case 'D':
90                 y[0] += UNIT_SIZE;
91                 break;
92             case 'L':
93                 x[0] -= UNIT_SIZE;
94                 break;
95             case 'R':
96                 x[0] += UNIT_SIZE;
97                 break;
98         }
99     }
100
101     public void checkApple() { 1 usage
102         if (x[0] == appleX && y[0] == appleY) {
103             bodyParts++;
104             applesEaten++;
105             newApple();
106             applesEaten++;
107             newApple();
108         }
109     }
110
111     public void checkCollisions() { 1 usage
112         for (int i = bodyParts; i > 0; i--) {
113             if (x[0] == x[i] && y[0] == y[i]) {
114                 running = false;
115             }
116         }
117
118         if (x[0] < 0 || x[0] >= SCREEN_WIDTH ||
119             y[0] < 0 || y[0] >= SCREEN_HEIGHT) {
120             running = false;
121         }
122
123         if (!running) timer.stop();
124     }

```

```

124
125 @ public void gameOver(Graphics g) { 1 usage
126
127     g.setColor(Color.red);
128     g.setFont(new Font( name: "Arial", Font.BOLD, size: 50));
129     FontMetrics metrics1 = getFontMetrics(g.getFont());
130
131     g.drawString( str: "Game Over",
132                 x: (SCREEN_WIDTH - metrics1.stringWidth("Game Over")) / 2,
133                 y: SCREEN_HEIGHT / 2 - 50);
134
135     g.setColor(Color.white);
136     g.setFont(new Font( name: "Arial", Font.BOLD, size: 30));
137     FontMetrics metrics2 = getFontMetrics(g.getFont());
138
139     g.drawString( str: "Final Score: " + applesEaten,
140                 x: (SCREEN_WIDTH - metrics2.stringWidth("Final Score: " + applesEaten)) / 2,
141                 y: SCREEN_HEIGHT / 2);
142
143     g.setFont(new Font( name: "Arial", Font.PLAIN, size: 20));
144     g.drawString( str: "Press R to Replay",
145                 x: (SCREEN_WIDTH - metrics2.stringWidth("Press R to Replay")) / 2,
146                 y: SCREEN_HEIGHT / 2 + 40);
147 }
148
149 @Override
150 public void actionPerformed(ActionEvent e) {
151
152     if (running) {
153         move();
154         checkApple();
155         checkCollisions();
156     }
157     repaint();
158 }
159
160 public void restartGame() { 1 usage
161     bodyParts = 6;
162     applesEaten = 0;
163     direction = 'R';
164     running = false;

```

```

166         for (int i = 0; i < x.length; i++) {
167             x[i] = 0;
168             y[i] = 0;
169         }
170
171         startGame();
172     }
173
174     public class MyKeyAdapter extends KeyAdapter { 1 usage
175
176         @Override
177         public void keyPressed(KeyEvent e) {
178
179             switch (e.getKeyCode()) {
180
181                 case KeyEvent.VK_LEFT:
182                     if (direction != 'R') direction = 'L';
183                     break;
184
185                 case KeyEvent.VK_RIGHT:
186                     if (direction != 'L') direction = 'R';

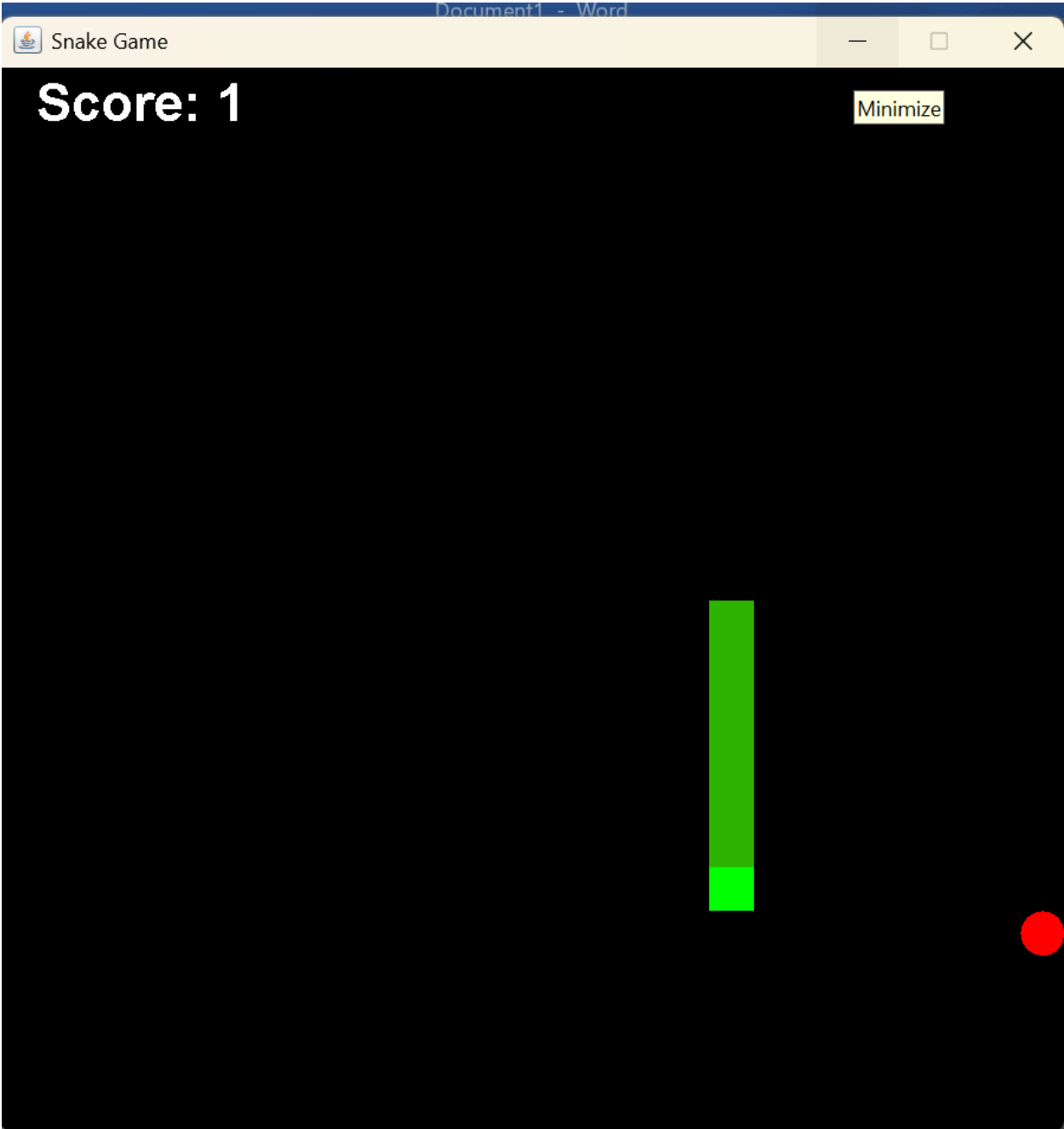
```

```

85                 case KeyEvent.VK_RIGHT:
86                     if (direction != 'L') direction = 'R';
87                     break;
88
89                 case KeyEvent.VK_UP:
90                     if (direction != 'D') direction = 'U';
91                     break;
92
93                 case KeyEvent.VK_DOWN:
94                     if (direction != 'U') direction = 'D';
95                     break;
96
97                 case KeyEvent.VK_R:
98                     if (!running) restartGame();
99                     break;
100             }
101         }
102     }
103 }

```

OUTPUT





Snake Game



Game Over

Final Score: 1

Press R to Replay